

Assignment 2 :

GitHub Link :

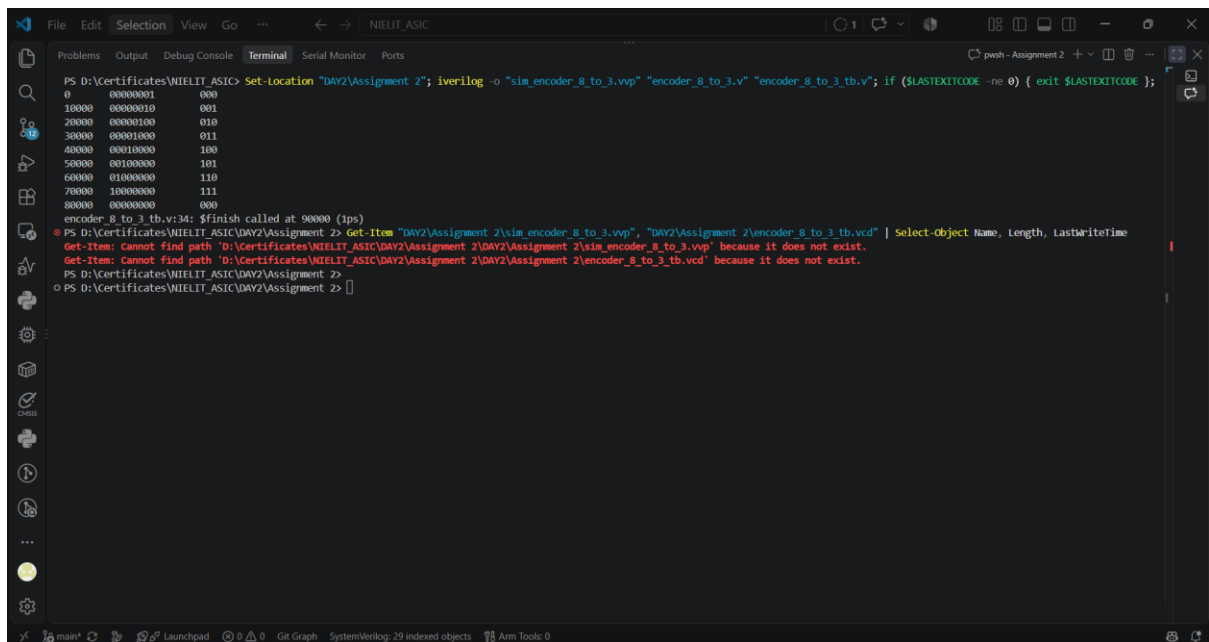
1. Link :
https://github.com/ssbagi/IBM_OpenPOWER_and_SoC_Design/tree/main/DAY2/Assignment%202
2. README.md file :
https://github.com/ssbagi/IBM_OpenPOWER_and_SoC_Design/blob/main/DAY2/Assignment%202/README.md

Question 1 : Design and Simulate 8:3 Encoder. Simulate all possible input combinations and verify the output.

Verilog code :

```
module encoder_8_to_3 (  
    input [7:0] in,  
    output reg [2:0] out  
);  
    always @(*) begin  
        case (in)  
            8'b00000001: out = 3'b000;  
            8'b00000010: out = 3'b001;  
            8'b00000100: out = 3'b010;  
            8'b00001000: out = 3'b011;  
            8'b00010000: out = 3'b100;  
            8'b00100000: out = 3'b101;  
            8'b01000000: out = 3'b110;  
            8'b10000000: out = 3'b111;  
            default: out = 3'b000;  
        endcase  
    end  
endmodule
```

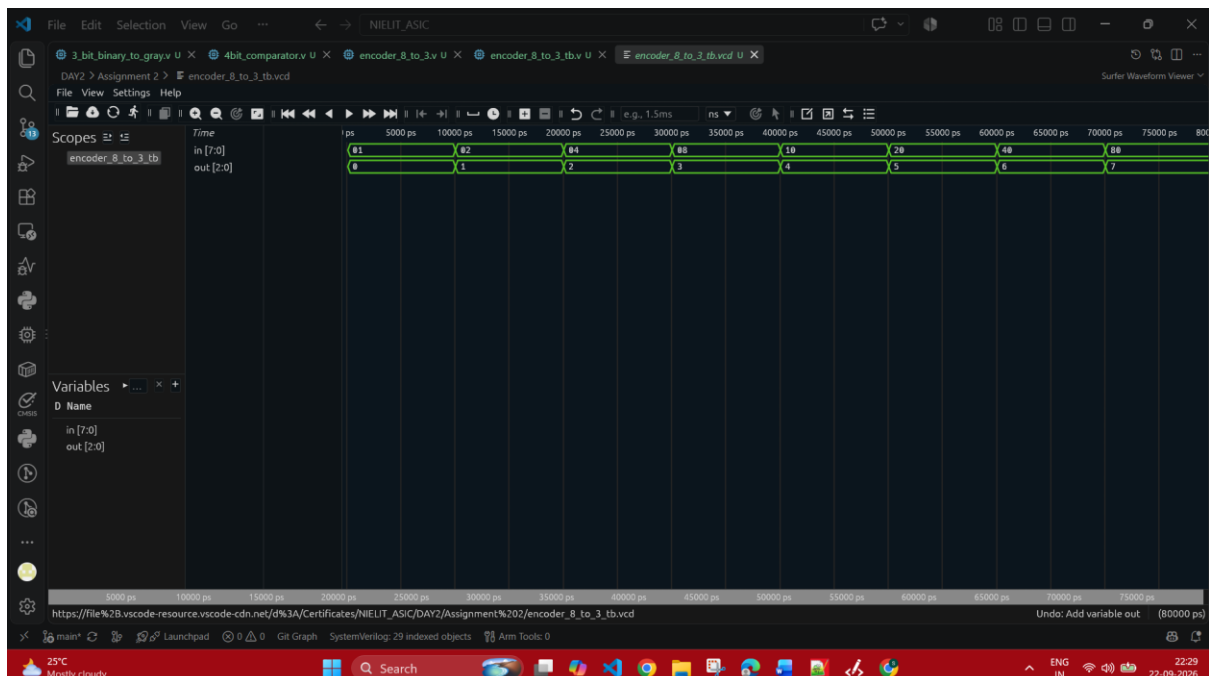
Compilation of the codes :



```
PS D:\Certificates\WIELIT_ASIC> Set-Location "DAY2\Assignment 2"; iverilog -o "sim_encoder_8_to_3.vvp" "encoder_8_to_3.v" "encoder_8_to_3_tb.v"; if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE };
0 00000001 000
10000 00000010 001
20000 00000100 010
30000 00001000 011
40000 00010000 100
50000 00100000 101
60000 01000000 110
70000 10000000 111
80000 00000000 000
encoder_8_to_3_tb.v:34: $finish called at 90000 (1ps)
PS D:\Certificates\WIELIT_ASIC\DAY2\Assignment 2> Get-Item "DAY2\Assignment 2\sim_encoder_8_to_3.vvp", "DAY2\Assignment 2\encoder_8_to_3_tb.vcd" | Select-Object Name, Length, LastWriteTime
Get-Item: Cannot find path 'D:\Certificates\WIELIT_ASIC\DAY2\Assignment 2\DAY2\Assignment 2\sim_encoder_8_to_3.vvp' because it does not exist.
Get-Item: Cannot find path 'D:\Certificates\WIELIT_ASIC\DAY2\Assignment 2\DAY2\Assignment 2\encoder_8_to_3_tb.vcd' because it does not exist.
PS D:\Certificates\WIELIT_ASIC\DAY2\Assignment 2>
PS D:\Certificates\WIELIT_ASIC\DAY2\Assignment 2>
```

Simulation Waveforms :

The Input values shown are in the Hexadecimal. The output selection is based on the inputs and stuff.



Question 2 :

Design and simulate 4-bit magnitude comparator that generates outputs indicating:

- $A > B$
- $A = B$
- $A < B$

Verilog code :

```
module comparator_4bit (  
    input [3:0] a,  
    input [3:0] b,  
    output reg a_greater,  
    output reg b_greater,  
    output reg equal  
);  
  
    always @(*) begin  
        a_greater = 1'b0;  
        b_greater = 1'b0;  
        equal = 1'b0;  
  
        if (a[3] != b[3]) begin  
            a_greater = a[3];  
            b_greater = b[3];  
        end else if (a[2] != b[2]) begin  
            a_greater = a[2];  
            b_greater = b[2];  
        end else if (a[1] != b[1]) begin  
            a_greater = a[1];  
            b_greater = b[1];  
        end  
    end  
endmodule
```

```

end else if (a[0] != b[0]) begin

    a_greater = a[0];
    b_greater = b[0];

end else begin

    equal = 1'b1;

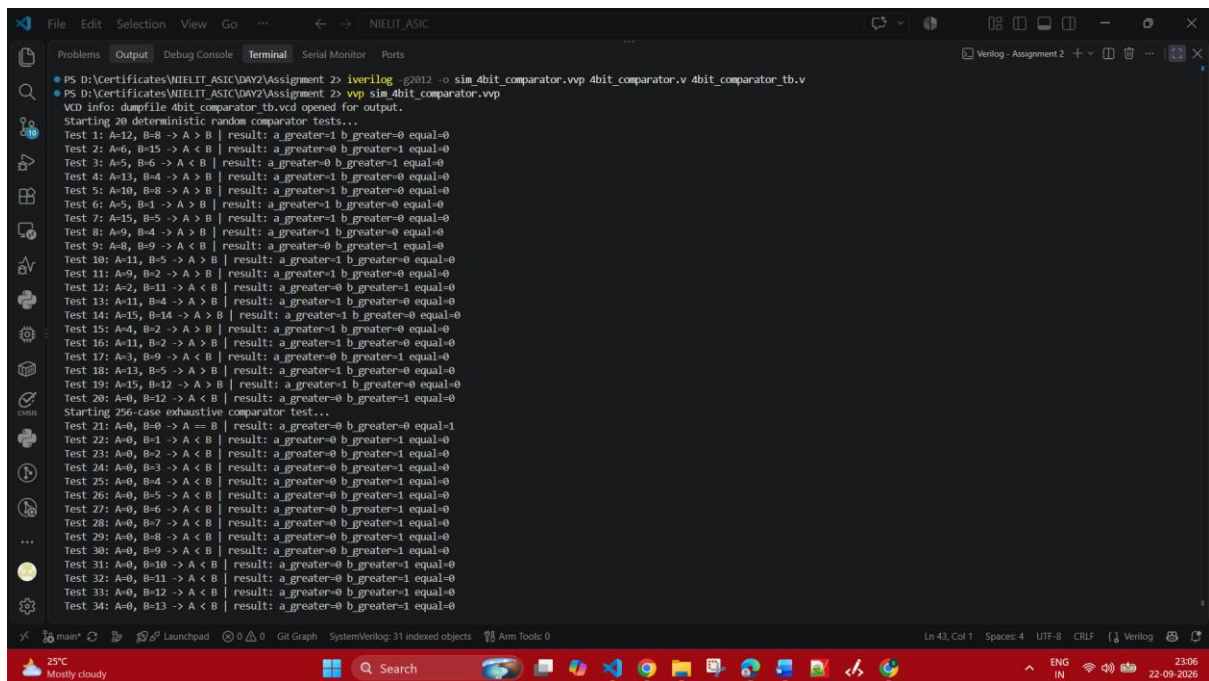
end

end

endmodule

```

Compilation :



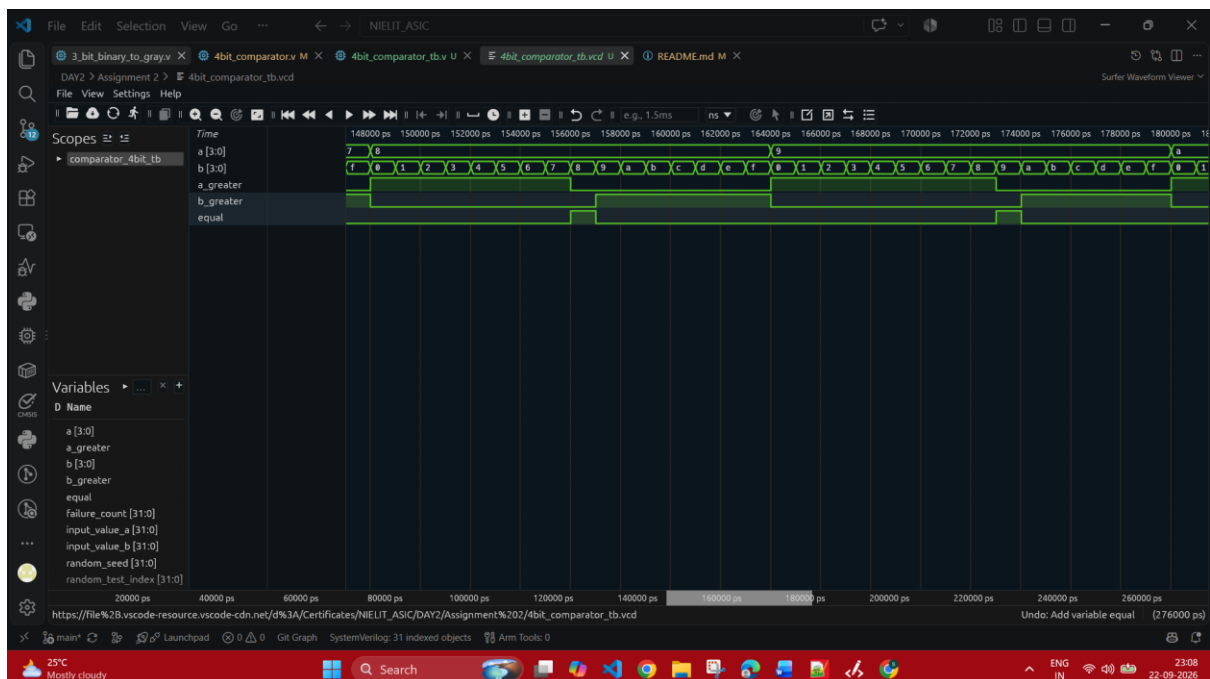
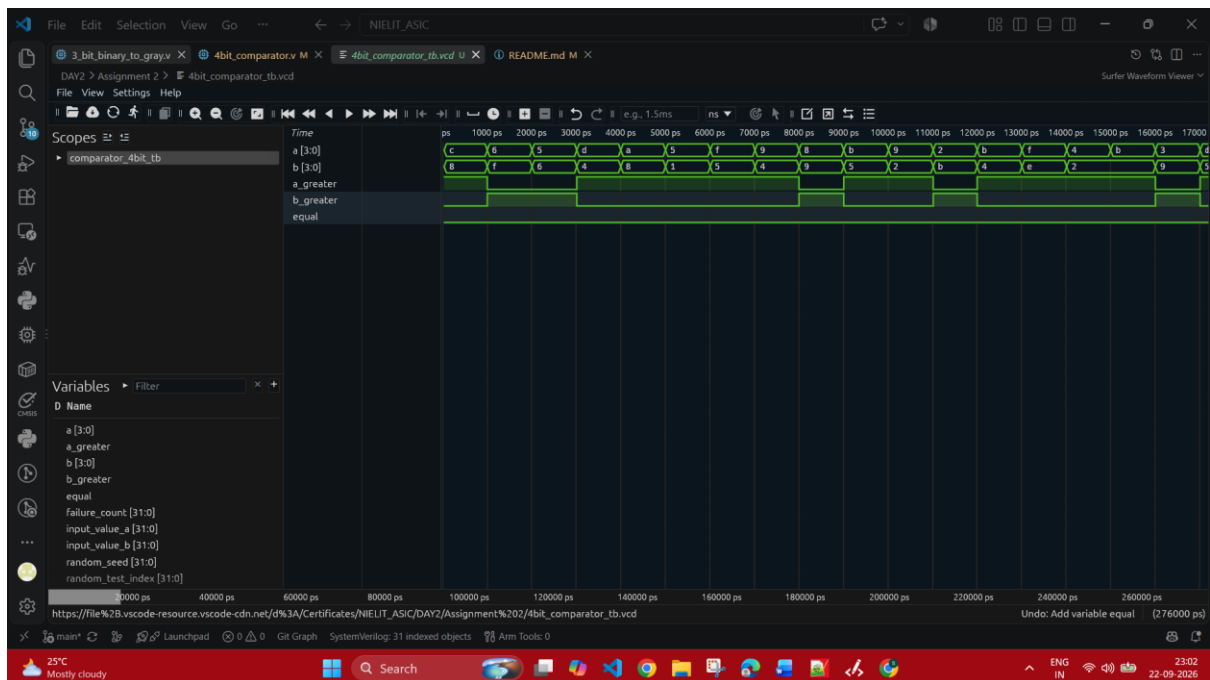
The screenshot shows a Verilog compilation window with the following content:

```

PS D:\Certificates\NIELIT_ASIC\DAY2\Assignment 2> iverilog -g2012 -o sim_4bit_comparator.vvp d4bit_comparator.v d4bit_comparator_tb.v
PS D:\Certificates\NIELIT_ASIC\DAY2\Assignment 2> vvp sim_4bit_comparator.vvp
VCD info: dumpfile d4bit_comparator_tb.vcd opened for output.
Starting 20 deterministic random comparator tests...
Test 1: A=12, B=8 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 2: A=6, B=15 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 3: A=5, B=6 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 4: A=13, B=4 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 5: A=10, B=8 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 6: A=5, B=1 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 7: A=15, B=5 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 8: A=9, B=4 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 9: A=8, B=9 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 10: A=11, B=5 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 11: A=9, B=2 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 12: A=2, B=11 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 13: A=11, B=4 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 14: A=15, B=14 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 15: A=4, B=2 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 16: A=11, B=2 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 17: A=3, B=9 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 18: A=13, B=5 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 19: A=15, B=12 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 20: A=0, B=12 -> A < B | result: a_greater=0 b_greater=1 equal=0
Starting 256-case exhaustive comparator Test...
Test 21: A=0, B=0 -> A == B | result: a_greater=0 b_greater=0 equal=1
Test 22: A=0, B=1 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 23: A=0, B=2 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 24: A=0, B=3 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 25: A=0, B=4 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 26: A=0, B=5 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 27: A=0, B=6 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 28: A=0, B=7 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 29: A=0, B=8 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 30: A=0, B=9 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 31: A=0, B=10 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 32: A=0, B=11 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 33: A=0, B=12 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 34: A=0, B=13 -> A < B | result: a_greater=0 b_greater=1 equal=0

```

Simulation : 4-bit Comparator

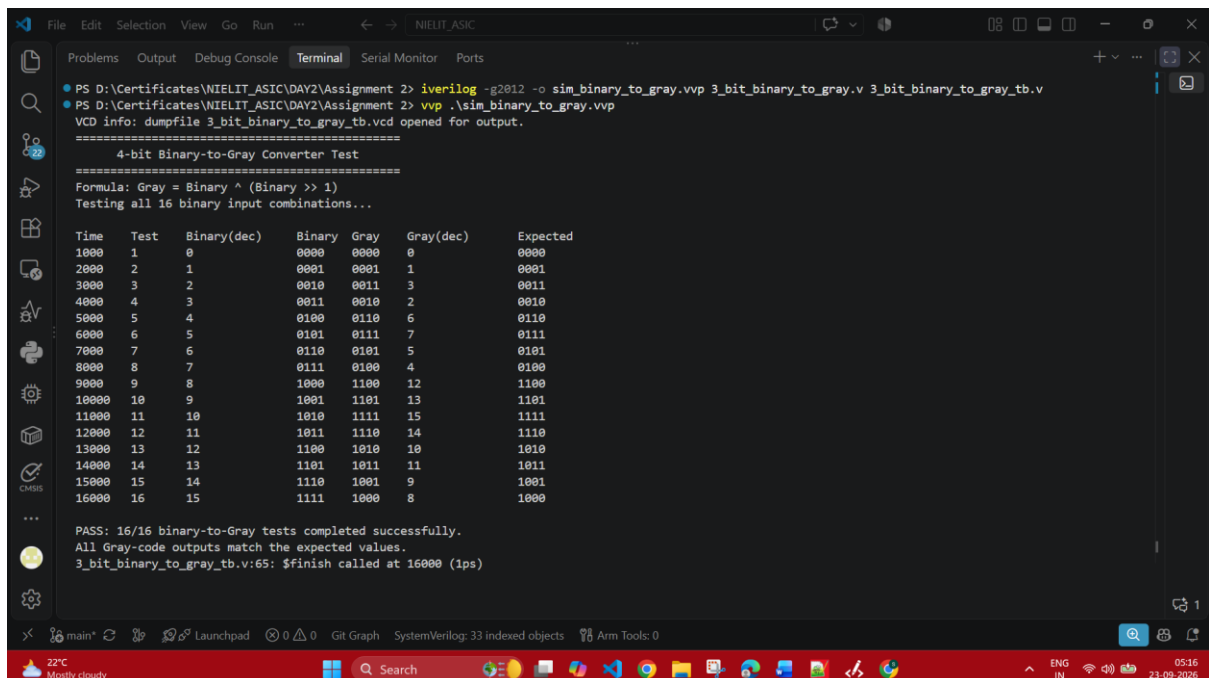


3-bit Binary to Gray Converter :

Verilog Code :

```
module binary_to_gray (  
    input [3:0] binary,  
    output [3:0] gray  
);  
  
    assign gray[3] = binary[3];  
    assign gray[2] = binary[3] ^ binary[2];  
    assign gray[1] = binary[2] ^ binary[1];  
    assign gray[0] = binary[1] ^ binary[0];  
  
endmodule
```

Compilation :



The screenshot shows a terminal window with the following content:

```
PS D:\Certificates\NIELIT_ASIC\DAY2\Assignment 2> iverilog -g2012 -o sim_binary_to_gray.vvp 3_bit_binary_to_gray.v 3_bit_binary_to_gray_tb.v  
PS D:\Certificates\NIELIT_ASIC\DAY2\Assignment 2> vvp .\sim_binary_to_gray.vvp  
VCD info: dumpfile 3_bit_binary_to_gray_tb.vcd opened for output.  
=====
```

4-bit Binary-to-Gray Converter Test

Formula: Gray = Binary ^ (Binary >> 1)
Testing all 16 binary input combinations...

Time	Test	Binary(dec)	Binary	Gray	Gray(dec)	Expected
1000	1	0	0000	0000	0	0000
2000	2	1	0001	0001	1	0001
3000	3	2	0010	0011	3	0011
4000	4	3	0011	0010	2	0010
5000	5	4	0100	0110	6	0110
6000	6	5	0101	0111	7	0111
7000	7	6	0110	0101	5	0101
8000	8	7	0111	0100	4	0100
9000	9	8	1000	1100	12	1100
10000	10	9	1001	1101	13	1101
11000	11	10	1010	1111	15	1111
12000	12	11	1011	1110	14	1110
13000	13	12	1100	1010	10	1010
14000	14	13	1101	1011	11	1011
15000	15	14	1110	1001	9	1001
16000	16	15	1111	1000	8	1000

PASS: 16/16 binary-to-Gray tests completed successfully.
All Gray-code outputs match the expected values.
3_bit_binary_to_gray_tb.v:65: \$finish called at 16000 (1ps)

Simulation :

